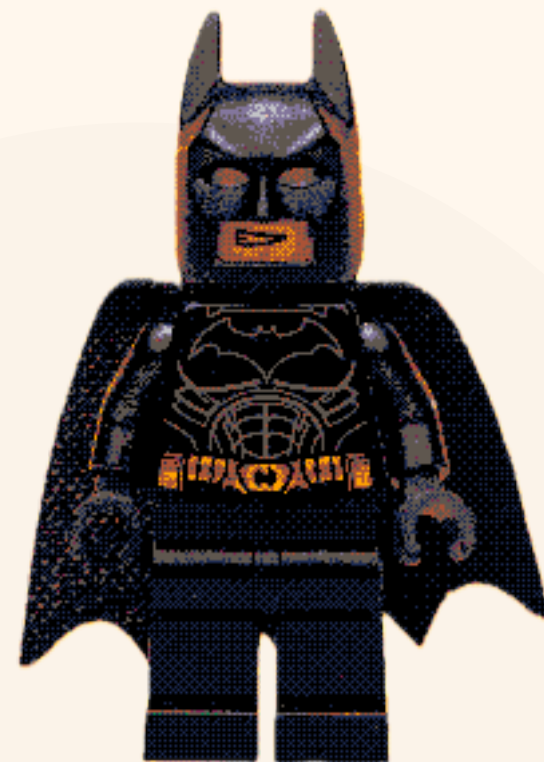


DevTools, Perf & Git

💡 Dagens fokus

- Forstå hvad der foregår i browseren når vores chatbots renderes
- Hvordan vi kan performance optimere dem
- Hvordan vi bedst bruger Git til at tracke ændringer.



Dagens program

12:00	5	🎓	Intro & agenda
12:05	20	💡	<u>Lav HTML strukturen til jeres AMAbots</u>
12:25	10	💡	<u>Diskuter semantisk opbygning af chatbot UI</u>
12:35	30	🛠️	<u>Byg chatbot UI i semantisk HTML og responsiv CSS</u>
13:05	15	🎓	<u>Git: Interaktion med remote repositories</u>
13:20	20	🎓	<u>Network requests i DevTools</u>
13:40	10	🕒	Pause
13:50	5	🎓	<u>Form submits som navigation</u>
13:55	20	🎓	<u>POST vs GET</u>
14:15	30	🛠️	<u>AMABot workshop</u>
14:45	10	🎓	Demo: Inline logo og CSS til ét request, perf analyse
14:55	20	💡	<u>Hands-on: Inline assets optimering</u>
15:15	0	🕒	Tak for i dag

1. Lav HTML strukturen til jeres AMAbots

Byg strukturen op sammen

Gennemgå Manuel Matuzović's nyligt opdaterede HTML boilerplate¹ med din sidemakker.

Diskuter hver sektion, han gennemgår:

01 Kendte I det pågældende tag eller teknik i forvejen?

Hvis ikke: Forklar den for hinanden.

02 Giver det mening for jeres AMAbot?

I så fald: Tilføj det til jeres `index.html`.

Parvis ⌚ 20 min

```
<!DOCTYPE html>
<html lang="en" class="no-js">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width">
  <meta name="text-scale" content="scale">

  <title>Unique page title - My Site</title>

  <script src="/js/render-blocking-scripts.js"></script>
  <script>
    document.documentElement.classList.replace('no-js', 'js');
  </script>

  <link rel="stylesheet" href="/css/styles.css">
  <link rel="stylesheet" href="/css/print.css" media="print">

  <script src="/js/non-render-blocking-script.js" type="module"></script>

  <link rel="icon" href="/favicon.ico">
  <link rel="icon" href="/favicon.svg" type="image/svg+xml">
  <link rel="apple-touch-icon" href="/apple-touch-icon.png">
  <link rel="manifest" href="/site.webmanifest">

  <link rel="canonical" href="https://web.site/page">
  <link rel="alternate" type="text/markdown" href="https://web.site/page.md">
  <link rel="me" href="https://mastodon.social/@username">
  <link rel="site.standard.publication" href="at://[DID]/site.standard.publication/[RKEY]">

  <meta name="description" content="Page description">
  <meta property="og:image" content="https://web.site/sm.jpg">
  <meta property="og:url" content="https://web.site/page">
  <meta name="theme-color" content="#FF5302">
  <meta property="fediverse:creator" content="@user@mastodon.social" >
</head>

<body>
  <!-- Content -->
  <script src="/js/non-render-blocking-script.js"></script>
</body>
</html>
```

¹<https://www.matuzo.at/blog/2026/html-boilerplate>

2. Diskutér semantisk opbygning af chatbot UI

🤝 Parvis ⌚ 10 min

Planlæg den strukturelle opbygning af jeres chatbot interface med fokus på semantisk HTML og brugervenlighed.

📁 Struktur og markup

- **Hovedcontainer:** `<main>` til chatbot-applikationen?
- **Beskedområde:** Fx `<section>` eller `<div class="messages">` til chat-historik?
- **Individuelle beskeder:** Fx `<article>` eller `<div class="message">` for hver besked?
- **Input-område:** `<form>` til besked-input og send-knap?
- **Navigation:** `<nav>` eller `<header>` til eventuelle menuer?

💡 Tilgængelighed og semantik

- **ARIA-labels:** Beskrivende labels for skærmlæsere?
- **Rolle-attributter:** `role="log"` til chat-historik for live-opdateringer?
- **Tastatur-navigation:** Tab-rækkefølge og fokus-håndtering?
- **Landmark-elementer:** Tydelig struktur for assistive teknologier?

3. Byg chatbot UI i semantisk HTML og responsiv CSS

🚩 Solo ⌚ 30 min

Oversæt jeres strukturelle overvejelser til kode.

📁 Implementér jeres struktur

- 01 **Start med HTML:** Opret en grundlæggende HTML-fil med jeres planlagte semantiske struktur
- 02 **Tilføj CSS:** Lav en separat CSS-fil og link den til jeres HTML
- 03 **Mobile first:** Design til små skærme først, udvid derefter
- 04 **Test løbende:** Preview filen, fx med Live Server² extension'en i VS Code

💡 Fokusområder under udviklingen

- **Semantisk HTML:** Brug passende elementer (`<main>` , `<section>` , `<form>` , `<article>`)
- **CSS Grid/Flexbox:** Moderne layout-teknikker til chat-container og beskeder
- **Responsive design:** Media queries til desktop vs mobile oplevelse
- **Tilgængelighed:** Kontrastforhold, fokus-states og ARIA-attributter

²<https://marketplace.visualstudio.com/items?itemName=ms-vscode.live-server>

3.1 Test responsive design på forskellige enheder

Sørg for at jeres chatbot virker godt på tværs af skærmstørrelser ved at teste systematisk.

Browser DevTools-testing:

- 01 **Åbn DevTools:** F12 eller højreklik → “Inspicer”
- 02 **Toggle device mode:** Klik på tablet/mobil-ikonet (Ctrl+Shift+M)
- 03 **Test forskellige størrelser:**
iPhone, iPad, desktop - både portrait og landscape
- 04 **Tjek touch targets:**
Er knapper store nok til mobil interaktion? (minimum 44px × 44px)

Hvad skal I teste:

- **Layout breakpoints:** Bryder design sammen ved bestemte bredder?
- **Tekstlæsarhed:** Er tekst for lille på mobile enheder?
- **Input-felter:** Fungerer tastatur-input godt på touch-enheder?
- **Scroll-adfærd:** Kan brugere scrolle gennem chat-historik naturligt?

Hvilken commit besked foretrækker I? 🤔



“first commit”



“Semantic HTML, accessibility and basic styling for initial version of chatbot”



“task”



“Lidt godter”

Hvilken commit besked foretrækker I? 🤔



“first commit”



“Semantic HTML, accessibility and basic styling for initial version of chatbot”



“task”



“Lidt godter”

Hvilken commit besked beskriver bedst hvad der er foregået? 🙊



“new commit”



“opg 3 færdig”



“Added sanitation for input, split data set and sanitation logic in util files.”



“small changes”

Hvilken commit besked beskriver bedst hvad der er foregået? 🙄



“new commit”



“opg 3 færdig”



“Added sanitation for input, split data set and sanitation logic in util files.”



“small changes”

4. Commit via GitHub Desktop, VS Code og Terminal

👤 Solo ⌚ 10 min

Lad os afprøve et commit workflow i tre forskellige værktøjer: GitHub Desktop, VS Code og Terminal.

💡 Hvorfor flere værktøjer?

→ Forskellige præferencer:

Nogle foretrækker visuelle interfaces, andre kommandolinjen

→ Kontekst-afhængig:

Terminal til scripting, GUI til kompleks merge-konfliktløsning

→ Læringsværdi:

Forståelse af underliggende Git-koncepter på tværs af værktøjer

📁 Vi gennemgår samme workflow:

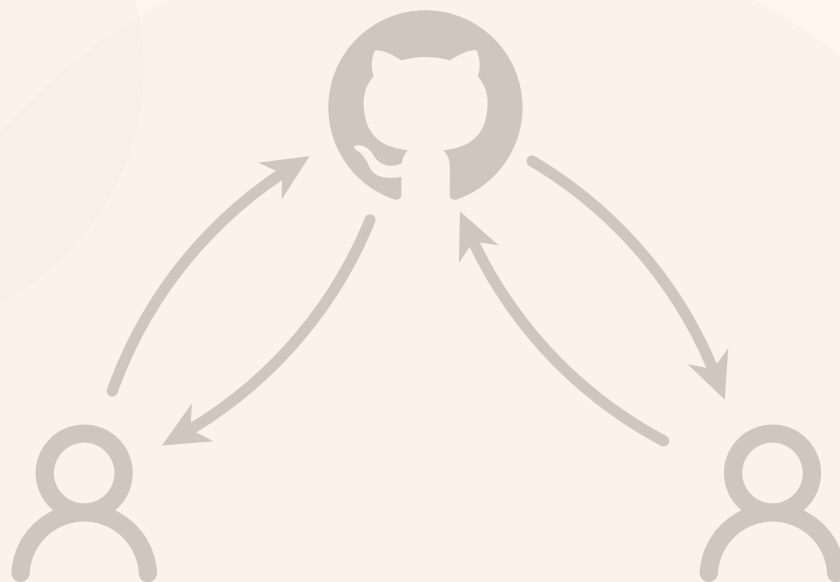
- 01 Lav en ændring i en fil i jeres chatbot projekt
- 02 Stage ændringerne til commit
- 03 Skriv en commit-besked og commit

5. Git: Interaktion med remote repositories

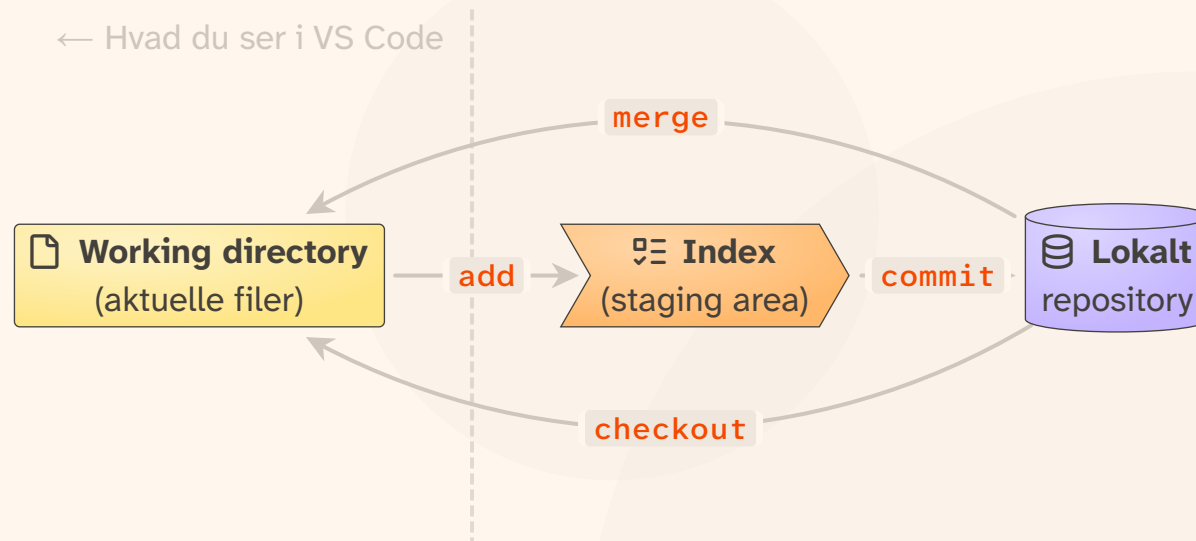
🔄 Hvad er et remote repository?

Et remote repository er en version af dit projekt, der er hostet på en server. Det giver mulighed for:

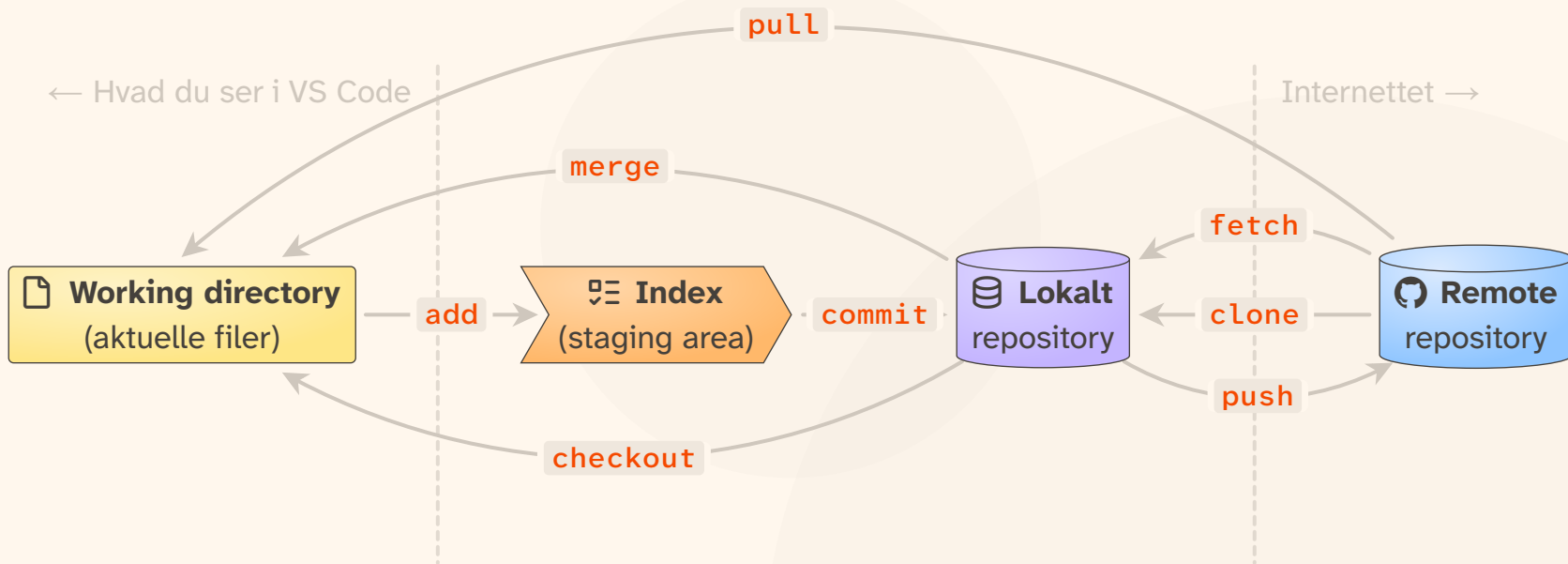
- **Samarbejde** mellem flere udviklere på samme projekt
- **Backup** af din kode på en ekstern server (f.eks. GitHub)
- **Versionsstyring** på tværs af forskellige maskiner
- **Distribution** af din kode til produktionsmiljøer (gennem et CI/CD workflow, der bruger Git)



5.1 Git workflow: lokalt



5.2 Git workflow: remote



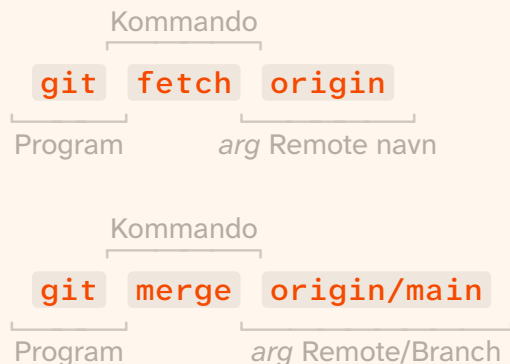
5.3 Fetch vs Pull: Hvad er forskellen?

💡 Fetch: Hent uden at merge

`git fetch` henter ændringer fra remote repository, men merger dem **ikke** automatisk:

- Downloader nye commits, branches og tags
- Opdaterer remote-tracking branches (f.eks. `origin/main`)
- Dit lokale arbejde forbliver uændret
- Giver dig mulighed for at inspicere ændringer før merge

📄 Fetch først, merge manuelt



📄 Pull: Fetch + Merge i én kommando



6. Network requests i DevTools

Network panelet i DevTools giver indsigt i hvordan jeres chatbot loader ressourcer og kommunikerer med serveren.

💡 Hvorfor analysere netværkstrafik?

- **Identificer flaskehalse:** Find hvilke ressourcer der tager længst tid om at loade
- **Forstå user experience:** Hvornår bliver siden brugbar for brugeren?
- **Optimér performance:** Find ud af hvilke optimeringer, der giver størst effekt
- **Debug problemer:** Track requests der fejler eller svarer langsomt

🔧 Åbn DevTools Network panelet:

- 01 Højreklik på jeres chatbot → “Inspect”
- 02 Gå til “Network” tab
- 03 Refresh siden for at se alle requests
- 04 Sorter efter “Waterfall” for at se timing

Elements Console Sources **Network** ⚠ Memory Performance Application >>

☐ Preserve log ☒ Disable cache Slow 4G

Filter ☐ Invert More filters ▼

All Fetch/XHR Doc CSS JS Font Img Media Manifest Socket Wasm Other

Name	Method	Status	Type	Size	Time	Waterfall
ask	POST	302 Found	document / Redirect	0.2 kB 0.0 kB	576 ms 575 ms	
localhost	GET	200 OK	document	3.6 kB 3.4 kB	589 ms 572 ms	
style.css	GET	200 OK	stylesheet	3.0 kB 2.6 kB	592 ms 574 ms	
robot-icon.png	GET	200 OK	png	5.9 kB 5.6 kB	613 ms 580 ms	

4 requests | 12.7 kB transferred | 11.6 kB resources | Finish: 1.21 s | DOMContentLoaded: 596 ms | Load: 1.22 s

6.1 Waterfall diagram og timing analyse

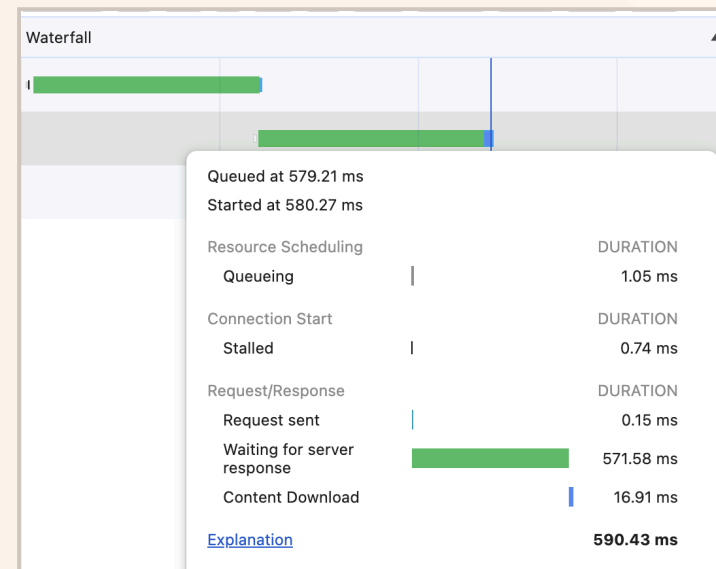
Waterfall diagrammet viser den præcise rækkefølge og timing af alle netværksanmodninger.

🕒 Timing faser i hver request:

- **Queueing:** Venter på tilgængelig forbindelse
- **DNS Lookup:** Opslag af serverens IP-adresse
- **Initial Connection:** Etablering af TCP-forbindelse
- **SSL/TLS:** HTTPS handshake (hvis relevant)
- **Request sent:** Selve HTTP-anmodningen sendes
- **Waiting for server response:**
Time To First Byte (TTFB) fra serveren
- **Content Download:** Downloading af response data

🔍 Analysér jeres chatbot:

- Hvilke filer loader først vs. sidst?
- Er der requests, der blokerer for andre?
- Hvor lang tid tager forskellige asset-typer?



6.2 Filtyper og performance

Forskellige ressourcer har forskellige performance-profiler og prioriteter i browseren.

📄 HTML og CSS:

- **Blokerende:** CSS blokerer rendering
- **Kritiske:** Nødvendige før siden kan vises
- **Størrelse:** Typisk små, men høj prioritet

📄 JavaScript:

- **Parser-blokerende:** Stopper HTML parsing
- **Async/defer:** Kan optimeres til ikke-blokerende
- **Størrelse:** Ofte nogle af de største filer

📄 Billeder og andre medier:

- **Ikke-blokerende:** Påvirker ikke initial render
- **Størrelse:** Typisk de tungeste filer
- **Lazy loading:** Kan udskydes til senere

📄 Fonts:

- **FOIT/FOUT:** Flash of Invisible/Unstyled Text
- **Preloading:** Kan optimeres med `<link rel="preload">`
- **Fallbacks:** System fonts som backup

6.3 HTTP headers og metadata analyse

Headers indeholder vigtig metadata om requests og responses, som påvirker caching, sikkerhed og performance.

💡 Vigtige request headers:

User-Agent Browser information

Accept Hvilke filtyper browseren kan håndtere

Accept-Encoding Komprimeringsmetoder (gzip, brotli)

Cache-Control Caching direktiver

Referer Hvor requesten kom fra

💡 Vigtige response headers:

Content-Type MIME-type af indholdet

Content-Length Størrelse i bytes

Cache-Control Server-side caching regler

ETag Version-identifier til cache validation

Content-Encoding Anvendt komprimering

🔍 Analysér i DevTools:

Klik på individuelle requests i Network panel for at se headers og timing detaljer

6.4 Performance implikationer og optimering

👤 Solo ⌚ 10 min

Brug DevTools Network panel til at identificere performance problemer i jeres chatbot.

🔍 Undersøg jeres chatbot:

- 01 Åbn DevTools Network panel og refresh siden
- 02 Sorter efter “Size” - hvilke filer er tungeste?
- 03 Sorter efter “Time” - hvilke tager længst tid?
- 04 Identificer blokerende ressourcer i waterfall

💡 Tænk over:

- Hvilke filer kunne optimeres eller fjernes?
- Er der unødvendige requests?
- Kunne noget loades async eller lazy?
- Er komprimering aktiveret på store filer?

7. Form submits som navigation

I traditionelle webapplikationer fungerer formularer som en af de primære måder at navigere mellem sider og sende data til serveren.

💡 Hvorfor forms som navigation?

- **Server-side routing:** Før AJAX og SPAs var forms den eneste måde at sende data til serveren
- **Siden reloades efter submit:** Browseren navigerer til en ny side eller genindlæser den nuværende
- **RESTful design:** Forms følger naturligt HTTP-metodernes formål (GET for søgning, POST for dataændringer)

📄 Eksempler

- **Login-formularer:** `POST` submit → omdirigering til dashboard
- **Søgefelter:** `GET`-request → resultatside med query parameters
- **Kommentar-formularer:** `POST` → reload med ny kommentar synlig
- **Checkout-flow:** Trinvis navigation gennem forskellige `POST` form-steps

7.1 Forms vs links: Hvornår bruger man hvad?

Forskellen mellem `<a>`-tags og `<form>`-elementer handler om intention og funktionalitet.

Links (`<a>` -tags):

- **Navigation uden dataændring:** Hent og vis eksisterende information
- **GET-requests:** Idempotente operationer (samme resultat hver gang)
- **Bogmærk-bar navigation:** URLs kan gemmes og deles
- **Eksempler:** Menuer, breadcrumbs, “læs mere”-links

Forms (`<form>` -elementer):

- **Dataindsamling og -transmission:** Brugers input skal sendes til serveren
- **POST/PUT/DELETE:** Operationer der ændrer data eller tilstand
- **Kompleksitet:** Flere forskellige input, validering, file uploads
- **Eksempler:** Login, søgning, bestillinger, brugerregistrering

I jeres chatbots:

- Beskeden sendes via form (POST), men navigation mellem sider kunne være links (GET)

8. POST vs GET

HTTP-metoder har forskellige formål og opfører sig forskelligt. Forståelse af POST og GET er afgørende for at bygge korrekte webapplikationer.

💡 Grundlæggende forskel:

- **GET** henter data fra serveren. Idempotent (gentagen anmodning giver samme resultat)
- **POST** sender data til serveren. Kan ændre serverens tilstand eller oprette nye ressourcer

💬 Diskutér i par:

- Hvordan tror I en søgning på Google fungerer? Og et login?
- Hvad sker der hvis I opdaterer siden efter et form submit? For henholdsvis GET og POST?

8.1 Praktisk forskel: URL og synlighed

POST og GET håndterer data meget forskelligt, hvilket har konsekvenser for brugeren.

GET-requests:

- **Data i URL:** Query parameters synlige i browserens adresselinje
- **Bogmærk-bar:** URLs kan gemmes og deles med andre
- **Browser-cache:** Resultater kan caches for bedre performance
- **Opdatering:** Tryk at gentage, ændrer ikke data på serveren

POST-requests:

- **Data i request body:** Ikke synligt i URL eller browserhistorik
- **Browser-advarsel:** “Vil du sende dataene igen?” ved genindlæsning
- **Ikke cacheable:** Browseren cacher ikke POST-responses som standard
- **Sideeffekter:** Kan ændre data, oprette brugere, sende emails osv.

Prøv det selv:

Åbn *DevTools* → *Netværk* og observer forskellen når I søger (GET) vs sender en besked (POST)

8.2 Identifikation af POST vs GET på websites

♥ Parvis ⌚ 10 min

Se om I kan spotte nogle GET og POST requests på forskellige websites, I bruger til daglig.

🔍 Sådan undersøger I:

- 01 **Åbn DevTools:** Højreklik → “Inspicer” → “Netværk”-fanen
- 02 **Ryd netværkslog:** Tryk refresh-ikonet i DevTools for en ren start
- 03 **Udfør handlinger:** Søg, log ind, send kommentarer, køb varer
- 04 **Observer requests:** Se “Method”-kolonnen i netværkstabelen
- 05 **Klik på requests:** Undersøg headers og payload for detaljer

💬 Find eksempler på:

- **GET:** Søgninger, navigation, indlæsning af sider
- **POST:** Login, kommentarer, formularer, køb
- **Blandet:** E-commerce sites bruger ofte både GET (produktvisning) og POST (add to cart)

8.3 GET vs POST og dataoverførsel

HTTP er grundlaget for al kommunikation mellem browser og server, og metoderne GET og POST håndterer data meget forskelligt.

💡 GET requests:

- **Datahentning:** Bruges til at hente information fra serveren
- **URL parametre:** Data sendes synligt i URL'en (`?name=value&other=value`)
- **Caching:** Browseren kan cache GET responses
- **Idempotent:** Samme request giver samme resultat hver gang

💡 POST requests:

- **Datasubmission:** Sender data til serveren for behandling
- **Request body:** Data gemmes “usynligt” i HTTP request body
- **Ikke cached:** Browseren cacher ikke POST responses
- **Side effects:** Kan ændre server-tilstand (oprette, opdatere, slette)

8.4 Hands-on analyse i DevTools

🔗 Parvis ⌚ 10 min

DevTools Network panel viser præcis hvordan jeres chatbot sender og modtager data.

🔍 Analysér jeres chatbot:

- 01 Åbn DevTools Network panel
- 02 Send en besked i jeres chatbot
- 03 Se hvilke HTTP metoder der bruges
- 04 Undersøg request/response headers og payload

🔍 Hvad skal I kigge efter:

- **“Method” kolonnen:** GET, POST eller andre HTTP verber
- **“Status” kolonnen:** 200 (success), 302 (redirect), 404 (not found), 500 (server error)
- **“Headers” tab:** Se request og response metadata
- **“Payload”/“Response” tab:** Faktisk data der sendes frem og tilbage

💬 Diskutér med hinanden:

- Hvilke requests bruger **GET** vs **POST** i jeres chatbot?
- Kan I se chatbeskederne i Network panelet?
- Hvad sker der når I refresher siden?

8.5 Sikkerhed og best practices

Valget mellem GET og POST har vigtige sikkerhedsmæssige konsekvenser for jeres chatbot.

👍 GET best practices:

- **Public data:** Kun til data der må være synligt
- **Search queries:** Søgninger, filtrering, navigation
- **Bookmarkable:** URLs der skal kunne gemmes/deles
- **Cache-friendly:** Statisk indhold, API data

⚠️ GET faldgruber:

- **Sensitive data:** Passwords, personlige oplysninger
- **Server ændringer:** Brug aldrig GET til at slette/opdatere
- **URL længde:** Browsers har begrænsninger på URL størrelse

🔄 I jeres chatbot projekt:

- Chatbeskeder skal sendes via POST (ikke synlige i URL)
- Hentning af tidligere beskeder kan ske via GET

👍 POST best practices:

- **Form submissions:** Login, chatbeskeder, uploads
- **API operationer:** Create, Update, Delete operationer
- **Sensitive data:** Data der ikke må være synligt i URL
- **Store payloads:** Store datamængder eller filer

⚠️ POST overvejelser:

- **CSRF protection:** Beskyttelse mod cross-site attacks
- **Duplicate submissions:** Undgå gentagne form submissions
- **Korrekt encoding:** Korrekt Content-Type headers

9. AMAbot workshop

🚩 Solo ⌚ 30 min

Brug tid på at bygge videre på din AMAbot og implementere de performance forbedringer, vi har gennemgået.

🔗 Tjekliste for Git forbedringer:

- ✓ **Commit audit:** Gennemgå eksisterende commits og identificer forbedringsmuligheder
- ✓ **Staging:** Øv dig i selektive `git add` kommandoer
- ✓ **Commit beskeder:** Skriv mindst 3 nye commits med korrekt beskedformat
- ✓ **Branching strategi:** Overvej om du skal bruge feature branches til nye funktioner

📄 Commit besked template:

[type]: Kort beskrivelse (under 50 tegn)

Længere forklaring af ændringen og hvorfor den var nødvendig.

- Specifik ændring 1
- Specifik ændring 2
- Overvejelser og noter

10. Hands-on: Inline assets optimering

🚩 Solo ⌚ 20 min

Afprøv de demonstrerede teknikker i jeres egne chatbot projekter.

🔄 Start en ny branch:

→ Opret en feature branch: `git checkout -b perf-inline-assets`

📄 Implementer i jeres chatbot:

- 01 **Logo optimering:** Konverter jeres logo til inline SVG eller base64 data URI
- 02 **Kritisk CSS:** Identificer og flyt above-the-fold CSS (eller al CSS) inline i `<head>`
- 03 **Performance baseline:** Mål load times i DevTools før ændringer
- 04 **Mål resultater:** Sammenlign performance efter optimeringerne

⌚ Fokusér på:

- **Network requests:** Færre HTTP requests = hurtigere load (?)
- **Filstørrelse trade-offs:** Inline øger HTML-størrelse men reducerer antal requests

